Estructuras de Datos Jerárquicas: Árboles Binarios

Fundamentos Matemáticos, Implementación en C y Aplicaciones en Ciencia de Datos

Jonnathan Ramos

Universidad Industrial de Santander — UIS
Escuela de Ingeniería de Sistemas e Informática
Ingeniería en Ciencia de Datos

6 de octubre de 2026

Motivación: De lo Lineal a lo Jerárquico

El Límite de las Estructuras Lineales

- **Búsqueda:** Arreglos y listas requieren $\mathcal{O}(n)$ comparaciones en el peor caso[cite: 2].
- **Inserción/Eliminación:** Mantener el orden exige desplazar elementos ($\mathcal{O}(n)$)[cite: 2].

El Salto Jerárquico

- Organización de datos por **niveles**[cite: 2].
- Reducción de la altura del espacio de búsqueda a $h \approx \log_2(n)$ en árboles balanceados[cite: 2].

Relevancia en Ingeniería y Ciencia de Datos

Versatilidad desde el *parsing* en compiladores y sistemas de archivos[cite: 3] hasta espacios de estados, *clustering* jerárquico y algoritmos de aprendizaje automático[cite: 2].

Grafo, Bosque y Árbol

- Un **Grafo** es un par $G = (V, E)$ donde V es el conjunto de vértices y $E \subseteq V \times V$ el conjunto de aristas[cite: 3].
- Un **Bosque** (*forest*) es un grafo acíclico[cite: 3].
- Un **Árbol** (*tree*) es un bosque conexo[cite: 3]. Sus vértices de grado uno se denominan **hojas**[cite: 3].
- Un **Árbol Plantado** (*rooted tree*) (T, r) posee un vértice r diferenciado llamado **raíz**[cite: 3].

Teorema de Equivalencia para Árboles

Para un grafo $T = (V, E)$, las siguientes afirmaciones son equivalentes:

1. T es un árbol.
2. Dos vértices cualesquiera están unidos por un **único camino**.
3. T es **mínimamente conexo** (eliminar cualquier arista lo desconecta).
4. T es **maximalmente acíclico** (añadir cualquier arista crea un ciclo).

Definición Inductiva y Recursiva

Definición Inductiva

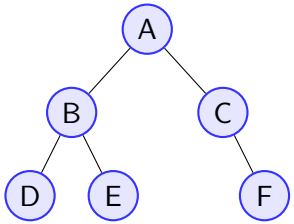
- **Base:** Un nodo n es un árbol (con raíz n).
- **Inducción:** Sea r un nuevo nodo y $T_1, T_2, \ldots, T_k$ árboles con raíces $c_1, c_2, \ldots, c_k$. Se construye un nuevo árbol T haciendo a r la raíz y añadiendo aristas desde r hacia cada c_i (sus hijos).

Definición Conjuntista / Recursiva

Un árbol es un conjunto finito de nodos tal que:

1. Existe un nodo especialmente designado llamado **raíz**.
2. Los restantes nodos están divididos en $m \geq 0$ conjuntos disjuntos $T_1, T_2, \ldots, T_m$, donde cada T_i es a su vez un árbol (**subárboles** de la raíz).

Anatomía y Terminología Esencial



- **Raíz / Hoja:** `A` es la raíz[cite: 2, 3]; `D`, `E`, `F` son hojas (nodos terminales, grado 0).
- **Padre / Hijo / Hermano:** `A` es padre de `B` y `C`[cite: 3]; `B` y `C` son hermanos.
- **Camino y Longitud:** Secuencia de nodos a recorrer; la longitud es el número de aristas del camino.
- **Nivel / Profundidad:** Número de ramas desde la raíz hasta el nodo].
- **Altura (h):** Longitud del camino más largo desde la raíz a una hoja ($h = 2$).
- **Ancestros / Descendientes:** Relación jerárquica extendida a lo largo del camino.

Árboles Binarios y Propiedades Matemáticas

Definición de Árbol Binario

Es un árbol en el cual cada nodo tiene **a lo más dos hijos**, denominados explícitamente **hijo izquierdo** e **hijo derecho**.

Propiedades Fundamentales

- **Grado máximo de cualquier nodo:** 2.
- **Número máximo de nodos en el nivel i:** 2^i (asumiendo nivel raíz $i = 0$).
- **Número máximo de nodos en un árbol de altura h:** $N_{\text{máx}} = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1$.
- **Cota de altura para N nodos:** $h \geq \lfloor \log_2(N) \rfloor$.

Clasificación de Árboles Binarios

Tipo	Descripción	Importancia Algorítmica
Lleno (Full)	Todo nodo no terminal tiene exactamente 2 hijos.	Codificación de Huffman.
Completo (Complete)	Niveles llenos salvo el último, llenado de izq. a der.	Base fundamental de *Heaps*.
Perfecto (Perfect)	Todos los niveles totalmente llenos ($N = 2^{h+1} - 1$).	Balance teórico ideal.
Balanceado	La diferencia de altura entre subárboles no supera 1.	Búsqueda garantizada $\mathcal{O}(\log n)$.

Representación en Memoria (C)

1. Representación Dinámica (Punteros)

```c
typedef struct NodoArbol {
    int dato;                       //
        Payload
    struct NodoArbol* izquierda; // Hijo
        Izq
    struct NodoArbol* derecha;   // Hijo
        Der
} NodoArbol;
```

Permite estructuras dinámicas no acotadas y de forma arbitraria.

2. Representación Secuencial (Arreglos)

Para **Árboles Completos** (*Heaps*):

- Dado un índice i:
- **Hijo Izquierdo:** $2i + 1$
- **Hijo Derecho:** $2i + 2$
- **Padre:** $\lfloor (i - 1)/2 \rfloor$

Recorridos en Profundidad (DFS)

Procesamiento recursivo aprovechando la **Pila de llamadas (*call stack*)**:

Pre-Orden

Raíz $\rightarrow$ **Izq** $\rightarrow$ **Der**

- Clonación y serialización de estructuras.
- Exportación de expresiones.

In-Orden

Izq $\rightarrow$ **Raíz** $\rightarrow$ **Der**

- En Árboles BST, produce la secuencia **ordenada ascendentemente**.

Post-Orden

Izq $\rightarrow$ **Der** $\rightarrow$ **Raíz**[cite: 2]

- Destrucción segura del árbol con `free()`[cite: 2].
- Evaluación de expresiones.

Gestión de Memoria en C

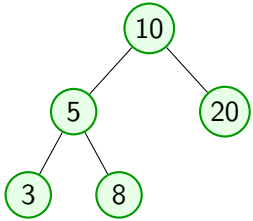
Liberar la raíz antes que sus subárboles (Pre-Orden) provoca *dangling pointers* y pérdidas de referencia (*memory leaks*). Siempre use **Post-Orden** para liberar.

Recorrido por Niveles (BFS)

- Visita los nodos **nivel por nivel**, de izquierda a derecha[cite: 2].
- Requiere una **Cola (Queue)** como estructura auxiliar explícita[cite: 2].

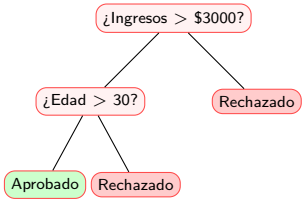
Algoritmo BFS

1. Encolar la raíz (`enqueue`)[cite: 2].
2. Mientras la cola no esté vacía:
   - Desencolar nodo (`dequeue`) y procesar[cite: 2].
   - Encolar hijo izquierdo (si existe)[cite: 2].
   - Encolar hijo derecho (si existe)[cite: 2].



Salida BFS: `10, 5, 20, 3, 8`[cite: 2]

Árboles Binarios en Ciencia de Datos



- **Árboles de Decisión (CART / XGBoost / Random Forests):** Reglas de decisión binarias sobre atributos; hojas representan clases o valores de regresión[cite: 2].
- **Montículos (Heaps / Filas de Prioridad):** Algoritmo de Dijkstra, selección eficiente de K-elementos superiores ($Top\text{-}K$)[cite: 2].
- **Indexación de Bases de Datos (BST / AVL):** Búsqueda de registros en $\mathcal{O}(\log n)$[cite: 2].

Conclusiones y Próxima Sesión

Puntos Clave

- Los árboles reducen la complejidad de búsqueda de $\mathcal{O}(n)$ a $\mathcal{O}(\log n)$ mediante su estructura jerárquica[cite: 2].
- La teoría de grafos provee los teoremas de aciclicidad y conectividad que sustentan su correcto funcionamiento[cite: 3].
- La implementación en C combina punteros dinámicos (‘struct‘) para flexibilidad y arreglos para eficiencia (*Heaps*)[cite: 2].
- DFS utiliza la pila implícita de la recursión; BFS requiere una cola explícita[cite: 2].

Próxima Sesión

Árboles Binarios de Búsqueda (BST): Algoritmos de inserción, eliminación, búsqueda $\mathcal{O}(\log n)$ y criterios de autobalanceo (AVL).